TLS-Fingerprint Scoping of Application-Layer DDoS Mitigation: Calibration Floors and Collateral on a CDN Operator's Endpoints

Natanael Oliveira*, Arthur Kobielski, Marcos Luna, Luigi Carro, Jéferson Campos Nobre, and Eder John Scheid
Institute of Informatics, Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre, Brazil
*Also with Azion Technologies, Brazil
{natanael.oliveira, arthur.kobielski, marcos.luna, carro, jcnobre, ejscheid}@inf.ufrgs.br

***Abstract*—A low-rate distributed application-layer flood keeps every source below per-source thresholds, so what matters is which clients to block. We treat that choice as a calibrated statistical test over the TLS fingerprints of an endpoint's clients and measure what it stops and costs, on generated campaigns and 23 days of a CDN operator's traffic. On generated traffic, the obvious choice, the alarm's most common fingerprint, blocks 39% of legitimate traffic and no attacker once the botnet spans five TLS stacks. The test blocks 90% of a botnet on up to 25 new stacks with no observed collateral. On production traffic, the test and its volume trigger misfire together in 0.1% of attack-free windows on the days used to choose them, though each alone exceeds its 1% budget. These misfires block a median third of the window's clients. On 14 later pre-specified days their rate doubles, still within budget. Calibration has a price: legitimate client fleets raise a floor, so a botnet on rare fingerprints that real clients share is singled out only when large. A botnet on common fingerprints mostly escapes. On the busiest endpoint the trigger lets most of a small injected botnet through. Post hoc triggers recover part of that loss on those days. We specify the filter and its counts in an OWL ontology, since no exchange standard we examined expresses it in its core vocabulary. The results tell an operator where fingerprint scoping pays off and where to fall back to rate limits or challenges.**

***Index Terms*—security management, application-layer DDoS, mitigation policy, TLS fingerprinting, statistical calibration, ontology**

I. Introduction

Application-layer Distributed Denial of Service (DDoS) attacks span two extremes. At one end, HTTP/2 Rapid Reset (CVE-2023-44487) pushed mitigation peaks to 398 million requests per second [1], a volume that capacity and scrubbing absorb. At the other end sit low-rate distributed floods, this paper's target. *Distributed Slow HTTP DoS* [2] and rate-based variants that still hold connections open, such as HULK and GoldenEye [3], exhaust a server from many coordinated origins, each sending too little to cross any per-source threshold.

A stealthy campaign also makes each session look legitimate on the per-session signals that detectors use, such as connection duration and packet rate [4]. What still distinguishes it is the structure *between* sessions: the same client fingerprint recurring across many origins that converge on one endpoint. Acting on that structure is a management decision: *which* clients to block, on *what* evidence, and with a scope narrow enough to spare legitimate users. The operator also needs to know, before an attack, where no such filter can be built, so that it can fall back there to a rate limit or a challenge.

Existing detectors at most flag the attack and leave that decision open. Of 75 application-layer DDoS studies in one survey, 47% derive their features from a user's request stream and packet pattern [5]. Graph-based detectors preserve relations between network nodes but stop at a report [6]. Aggregate-based mitigations filter on network and transport headers [7], [8], source-based filtering weighs each blocked prefix against the legitimate traffic it carries [9], and standard mitigation channels filter on the same fields [10], [11]. We found no published method that scopes a mitigation by application-client fingerprint while quantifying each filter's collateral and identifying where no such filter can be built.

This paper turns that decision into a calibrated statistical test (Fig. 1). For each endpoint and five-minute window, a volume trigger first raises the alarm. The test then chooses the *scope*, the TLS fingerprints to block: those over-represented among the alarm's origins relative to a *profile* of the endpoint's normal traffic. We target botnets of compromised devices, whose TLS stacks the attacker cannot cheaply replace, and we evaluate bots that present common fingerprints as the method's boundary.

The paper makes three contributions. *(i) Profile-relative scoping.* On generated traffic, profile-relative scopes block most of the attack without labels, while the alarm's most common fingerprint blocks legitimate users and no attackers once the botnet spans several stacks. *(ii) A calibration method for the scope.* We calibrate the test per endpoint to a false-alarm budget of 1% of attack-free windows, which sets a *calibration floor*: the botnet size from which the test names a typical stack. We evaluate budget and floor on eight days across four CDN endpoints, and the budget on 15 later days. *(iii) A session ontology as the specification.* An OWL ontology specifies the counts the decision reads and the scope it exports, and the log store's query is compiled from it, so a new equality signal needs no code change. No exchange standard we examined expresses this JA4 filter in its core vocabulary.

II. Related Work

Table I compares prior mitigation scopes and detectors on what a filter matches, how it is chosen and what it costs

TABLE I
COMPARISON WITH RELATED WORK.

Approach	Filter matches	Chosen by	Cost to legitimate users
Aggregate rate control [7], [8]	Header-defined aggregate	Congestion or aggregate score	Rate-limited with the aggregate
Blackholing [12]	Destination prefix, or header fields	The attacked network	All its traffic, or less with finer filters
Source-based filtering [9]	Source prefixes	Optimization under a filter budget	Traffic of the filtered prefixes
Anomaly extraction [13], [14]	Flow feature values	Over-representation in anomalous traffic	None (diagnosis only)
Signature generation [15]	Payload tokens	Coverage against a normal pool	False positives on normal traffic
Puzzles [16]	Every client under attack	The cost a client pays	A puzzle each
DOTS, Flowspec [10], [11]	Network and transport fields	The requesting party	That of the filter carried
Detectors [4], [6], [17]	None (verdict only)	Profile or learned model	Detection FPR only
Bot management [18]	JA4	Undisclosed	Undisclosed
This work	TLS fingerprints (JA4)	Calibrated enrichment against the profile	Measured per alarm and window

legitimate users.

Scoping a mitigation. Aggregate-based congestion control identifies attack aggregates from header fields and rate-limits them upstream [7], and more recent work scores such aggregates in programmable switches [8]. Blackholing drops all traffic to the attacked prefix, while advanced blackholing narrows the match to header fields to limit collateral damage [12]. Source-based filtering selects the prefixes to block under a filter budget, trading attack traffic stopped against legitimate traffic carried by those prefixes [9]. Closer to our test, feature distributions detect and classify traffic anomalies [19], while anomaly extraction isolates feature values over-represented in anomalous traffic [13], [14], the same question we ask of fingerprints. Signature generation likewise checks its candidates against a pool of normal traffic [15]. Against attacks that mimic flash crowds, defenses either make every client solve a puzzle [16] or schedule each session by how far it strays from legitimate workload profiles, weighing the delay to legitimate users [20]. All of them work on network, flow, session or payload features, without the fingerprints of an application's clients. The IETF DOTS data channel [10] and BGP Flow Specification [11] likewise filter on network and transport fields and cannot express a fingerprint such as JA4.

Layer-7 detection. Flow-based and Layer-7 HTTP DDoS detectors [5] either profile traffic, using Principal Component Analysis (PCA) over flow statistics [21] or per-session behavioral matrices [17], or learn from session features [4]. None of these preserves relations across sessions. Flash crowds are told apart from floods by how requests spread over a site's documents [22].

Knowledge graphs and fingerprints. Security knowledge graphs link entities extracted from threat reports or map countermeasures to ATT&CK, as D3FEND does in OWL [23], while their operational use remains unclear [24]. Traffic-derived graphs also feed learned detectors: KLAGE [6], the closest to our setting, classifies nodes in graphs built from network logs. Passive TLS fingerprinting identifies client software from its *ClientHello* [25], while many applications built on one library often share a fingerprint [26]. That sharing makes a stack useful as a scope and also sets its limit. Most bots that claim a browser identity expose TLS and header fingerprints from another tool [27]. One bot-management platform aggregates signals per JA4 over recent traffic [18], but publishes no model, benchmark or collateral metric.

III. CALIBRATED FINGERPRINT SCOPING

This section defines the threat model, trigger, scope and calibration, the resulting floor, and the ontology specification (Fig. 1).

A. Threat Model

The defender operates the attacked application and observes, per request, the TLS *ClientHello*, source address with prefix and ASN, target endpoint, arrival time, and any identity material exposed by the application. From the *ClientHello* it computes a JA4 fingerprint [28], which summarizes the TLS version, cipher suites, extensions and ALPN. The defender sees neither the botnet's control channel nor which sources are coordinated, but can profile its own traffic outside attack episodes. The attacker controls K compromised devices and may make each session statistically indistinguishable from a legitimate one, the stealth assumption of our evaluation. Harm comes from exhausted capacity: each session holds a connection and worker as a legitimate one does, so no per-session rate rises.

We also assume a bot keeps the fingerprint of its TLS stack: a TLS-speaking bot uses the library shipped with its device or toolkit, and botnets of compromised devices span many router, camera and DVR models [29]. Presenting a browser's *ClientHello* requires a full browser or an impersonation library kept in step with browser releases [30]. Bots that do so, such as headless-browser farms or bots built on TLS-impersonation libraries, fall outside this model, and Section V evaluates them as its boundary.

B. The Trigger

For each endpoint e and five-minute window, let S be the sessions to e and n their distinct origins, that is, source addresses, each counted once per fingerprint it presents. The trigger fires when $n \geq k_{\min} = 5$ and n reaches its 99th percentile over attack-free windows. This **distinct-origin gate** is the trigger of the production evaluation. The base rule, the held-out protocol's reference, uses the coordination mass $\Omega(S)$ of the rule for the ontology's class `CoordinatedHTTPFlood`, with its threshold τ_{cluster} at the same percentile. Ω is a weighted count of pairs of distinct origins whose sessions share a JA4, endpoint or /24 prefix (Appendix A). It mostly

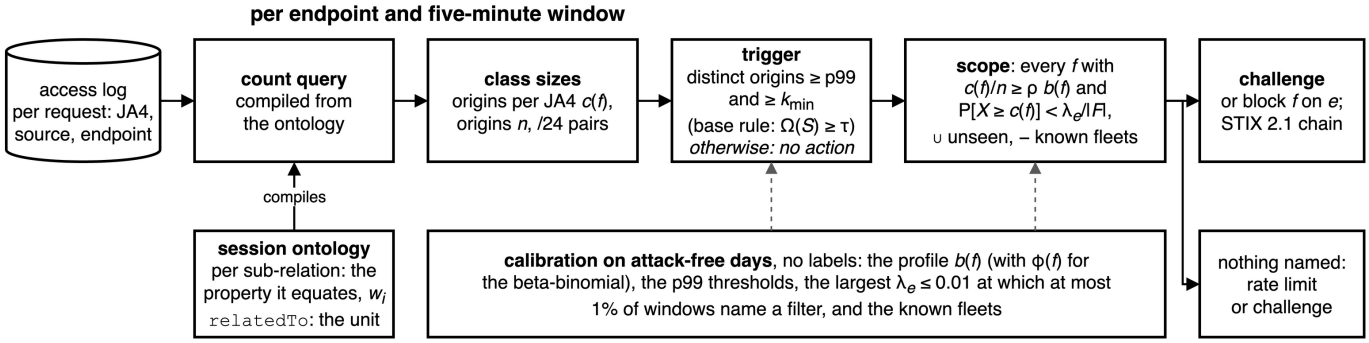


Fig. 1. Calibrated fingerprint scoping. The query compiled from the session ontology returns, per endpoint and window, the class sizes read by the trigger and the scope, and every threshold and level comes from attack-free days.

measures volume: its endpoint term, $0.6\binom{n}{2}$, contributes 91% of it in the default campaign (Listing 1, Appendix D). What separates the attack from legitimate traffic is the scope.

C. The Scope and Its Calibration

How the scope is chosen decides whether mitigation helps or harms. The obvious choice, the alarm's most common (modal) fingerprint, picks whatever is common, and on a service under attack that is the legitimate population. A botnet of A attackers spread uniformly over M stacks puts about $0.9A/M$ on each and the remaining tenth on one-off fingerprints, while the most common legitimate fingerprint holds a share p_1 of the n_b legitimate origins. The modal fingerprint is an attack stack only while $0.9A/M > p_1 n_b$. With the measured $p_1 = 38.4\%$ of requests (Section IV), taken as a proxy for origins, and a campaign as large as the legitimate traffic, this holds only while $M < 2.3$. With more stacks, each one smaller, the filter blocks users and no attackers.

We therefore compare the alarm with a **profile** of the endpoint's normal traffic, which the operator keeps outside attack episodes. Let $b(f)$ be $1/N$ plus the share of fingerprint f among the profile's N pooled origin–fingerprint pairs, $c(f)$ the number of the alarm's origins that present f, and $n = \sum_f c(f)$. The **binomial enrichment test** names every f that is both enriched, $c(f)/n \geq \rho\, b(f)$ with $\rho = 3$, and unlikely under the binomial background at level λ_e, $\Pr[X \geq c(f)] < \lambda_e/|F|$ for $X \sim \mathrm{Bin}(n, b(f))$. The Bonferroni divisor $|F|$ counts the fingerprints of the profile and the alarm. The result is a *set* of fingerprints, so it covers a botnet split over many stacks. A per-fingerprint z**-score** of $c(f)$ against $\mathrm{Bin}(n, b(f))$ provides a simpler profile-relative scope. The **unseen** filter names the fingerprints absent from the profile and seen in at least $k_{\min}$ origins, and needs no profile statistics. Joined with the test (the **union**), it catches new stacks too small for the level.

On generated traffic each origin carries one session, and the level λ_e stays at 0.01. On production traffic, *fleets* of legitimate clients switch on together and violate the binomial's independence. We therefore set λ_e per endpoint e to an empirical quantile: the largest value up to 0.01 at which the scope names a filter in at most 1% of the endpoint's attack-free calibration windows (the *budget*), about 2.9 of a day's 288 windows. At levels such as 10^{-60} the binomial tail no longer gives the probability of the data. It acts as a score whose threshold the quantile sets. The z-score's threshold follows the same budget, as the 99th percentile of each calibration window's largest z, never below 3. Both budgets hold on the calibration windows by construction, and Section V-B measures how well they hold on the days that follow.

The profile reflects what a defender sees in normal operation, so no attack label is required. It must exclude the attack episode, because a campaign spanning the whole window would otherwise give every candidate an enrichment of one. Conversely, attackers hidden inside a popular legitimate fingerprint do not enrich it, so the test leaves it unnamed, which is correct when nothing distinguishes the attack from legitimate traffic.

D. The Calibration Floor

The calibrated level has a price, the **calibration floor**: the botnet size from which the scope names a typical stack. Among n origins, the test names a stack of c attackers only if $c/n \geq \rho\, b(f)$ and $\Pr[X \geq c] < \lambda_e/|F|$. The floor is the smallest botnet of A attackers on M stacks whose average stack satisfies both conditions, with $0.9A/M$ attackers in a window of $n_0 + A$ origins, where n_0 is the median window of the calibration days. Lower levels raise the floor. In the union, the unseen filter names a stack absent from the profile once it holds $k_{\min}$ origins, requiring $\lceil k_{\min} M/0.9 \rceil$ attackers regardless of level, 139 for $M = 25$. The level's floor therefore matters on stacks that real clients also present, and there the ratio sets a harder limit. A stack's expected share of the window stays below $0.9/M$, so a botnet on M stacks is not expected to enrich a fingerprint whose share exceeds $0.9/(\rho M)$, 1.2% for $M = 25$ and $\rho = 3$.

We evaluate two ways to lower the floor. The first exempts the endpoint's *known fleets*, the fingerprints named at the nominal level in at least 5% of calibration windows, from the scope and from the calibration. The exemption is an allow-list for the busiest endpoint and the API, covering fingerprints that together carry up to about 7% of their profiles, so the scope never blocks a bot that presents a fleet's fingerprint. The second, built after the held-out day was read (Section V-D), is

a **beta-binomial background**, which allows within-window correlation. Its correlation $\phi(f)$ is estimated by moments per profile fingerprint over the calibration windows, from $\mathrm{Var}[c(f)] = nb(1-b)\big(1+(n-1)\phi\big)$, clipped to $[0, 0.99]$, and each count is tested against a beta-binomial with the same mean. A fingerprint absent from the profile remains binomial.

E. The Session Ontology as Specification

All counts above are specified in the session ontology, which is what an operator would deploy. Its central class, `ApplicationSession`, links a session to its `Identity`, source `IPAddress` and `Endpoint`. An `Attack` links to its `Mitigation`, whose scope the evidence chain carries. Sub-properties of a symmetric `relatedTo` type the evidence linking two sessions (Appendix A). The three used here are equalities: sharing a JA4, an endpoint or a /24 prefix partitions the sessions into classes. The ontology annotates each equality with the session property it compares, and `relatedTo` with the unit it counts in. From those annotations we compile the SQL queries that return, per window and endpoint, the class sizes read by the gate, base rule and scope, given only a property-to-column binding. No reasoner runs in the evaluation. A new equality signal reaches the compiled query through four triples plus a binding for any new session property, with no code change. When the rule fires, the evidence chain (the rule, cluster, scope and split of $\Omega(S)$ by sub-relation) is exported as JSON-LD (Listing 1, Appendix D) and STIX 2.1 [31], as an *indicator* and a *course-of-action* linked by *mitigates*.

IV. Evaluation Methodology

We use three traffic sources, each answering a different question. A generator, calibrated on benign CICIDS2017 sessions and on JA4 fingerprints measured at a CDN, runs the controlled campaigns. Laboratory captures test conventional attacks and allow an uncontrolled comparison with the published result of KLAGE, the closest system (Appendix C). The CDN's traffic, over 23 days, measures the scopes' false alarms and, with injected botnets, their detection.

A. Scenarios and Datasets

Controlled evaluation: calibrated synthetic traffic. A generator produces distributed Slow HTTP DoS campaigns against one endpoint with known ground truth. Its main parameter is the *degree of distribution* K, the number of devices controlled by the attacker. We report $K = 50$, a small botnet whose origins each stay below their own rate limit, and $K = 1000$, spread across hundreds of ASNs and prefixes, with almost no per-session signal. Legitimate fingerprint popularity follows a Zipf curve with exponent α, since real populations are concentrated. We compared it with 6.33M TLS requests logged at a production edge point of presence of Azion Technologies[1]: 495 distinct fingerprints, the most common carrying 38.4% of requests and the top ten 93.8%. The measured curve's head matches $\alpha = 1.5$ and its top ten $\alpha = 2.0$, and we use $\alpha = 1.5$ by default.

No published measurement fixes how botnets spread over stacks, so we vary it: 90% of the campaign spreads over M stacks uniformly at random, the hardest case in expectation for naming a stack at a given M, and the rest over one-off fingerprints. The default is $M = 25$. A *shared* mode draws the stacks from the profile's tail, which real clients also present, and an *adversarial* mode gives the botnet the most common legitimate fingerprints, as a bot imitating a browser would.

Legitimate request counts and durations are sampled from benign CICIDS2017 sessions (Appendix B), so Kolmogorov–Smirnov distances of $D = 0.002$ and 0.003 between generated and real sessions only check the sampler. In *stealth* mode each attacker session draws its attributes from the same distributions, so the campaign appears only in the structure across sessions. Attackers start in the first 30 s, making the campaign a large share of the traffic it overlaps: a median 34–45% of a window's origins and 50% of the alarm's *cluster*. The cluster chains the endpoint's sessions as long as each starts within 300 s of the previous one.

Production traffic. We use eight days of four endpoints of Azion's own services: real-user monitoring beacons (E1), a web console (E2), an API (E3) and single sign-on (E4), whose median windows hold about 3 000, 62, 41 and 20 origins. Of the 12 endpoints exported, they are the four that carry TLS and have at least 50 calibration windows and a median window of at least $k_{\min}$ origins. The exports count distinct origins and connections per fingerprint and window, plus same-/24 pairs per window. The scope is evaluated offline, behind the operator's WAF, so the profiles and floors describe the clients the WAF lets through. Calibration days are treated as attack-free: an attack there would lower the level and raise the gate's threshold, making both more cautious. Test-day windows before injection (*clean*) are taken as attack-free too, so a real attack there would count as a false alarm.

Each of the last five days is tested using the profile, thresholds and level from preceding days. Into each of its windows we inject a botnet built as the generator builds one. It holds 25 to 1 000 attackers, or from a tenth of the median window (*tenth-size*) to all of it. It puts 90% of its attackers on 25 stacks, uniformly at random, and the rest on one-off fingerprints that no scope names, so at most about 90% can be blocked. Its stacks are absent from the traffic (*new*), drawn from the profile past its ten most common fingerprints (*shared*), or the endpoint's 25 most common fingerprints (*adversarial*). A flash crowd adds 100 or 1 000 users drawn from the day's own clients to one window. A ninth day, the *held-out day*, and 14 more, the *block*, were analyzed as fixed in advance.

B. Baselines, Configurations and Metrics

On generated traffic we compare four scopes on the alarm's cluster: the modal fingerprint, the unseen filter, the z-score ($z > 3$) and the binomial test. The learned baseline is a Random Forest trained on the campaign's labels over the cross-session evidence of the three sub-relations (configuration (d)

[1]Measured August 22, 2026, from an access log the platform already collects. The extract holds only client-software fingerprints and their frequencies, with no address, host, URI, header or locale.

TABLE II
THE CONFIGURATIONS EVALUATED ON PRODUCTION TRAFFIC.

Configuration	Gate	Scope	Background	Exempt	Chosen	Held-out day
Base rule	$\Omega \geq \tau$	test	binomial	–	test days	in protocol
Binomial	origins	test ∪ unseen	binomial	fleets	test days	in protocol
z-score	origins	$z >$ its p99	binomial	–	test days	outside[a]
Beta-binomial	origins	test ∪ unseen	beta-bin.	–	post hoc	post hoc*

*Post hoc: built after the held-out day was read. [a]In the code before the day was read, but outside its protocol.

TABLE III
SCOPES OF THE ALARM'S CLUSTER AS DETECTORS.

	Binomial test			Other scopes			Learned, FPR$= 0$	
Scenario	Recall	FPR	F_1	Modal	z-score	Unseen	(d)	+ profile
M=1	89.8%	0.00%	0.946	89.8%	89.8%	89.8%	91.7%	85.9%
M=5	90.0%	0.00%	0.948	0.0%	90.0%	90.0%	88.6%	91.7%
M=25	90.3%	0.00%	**0.949**	0.0%	90.3%	90.3%	36.4%	87.4%
M=100	38.6%	0.00%	0.556	0.0%	80.4%	88.1%	17.6%	86.9%
M=25, shared	85.4%	2.23%	0.910	0.0%	89.9%	0.0%	–	–
M=25, adversarial	30.4%	3.78%	0.452	3.6%	57.3%	0.0%	7.8%	6.6%

Recall of each scope alone, $K = 1000$, 15 seeds. Collateral of the other scopes: Section V-A. The learned models were not run on shared stacks.

of Appendix C). It is held to zero false positives, which the test reaches wherever the learned model runs except the adversarial case. On production traffic we evaluate the configurations of Table II. Their gate is either $\Omega \geq \tau_{\text{cluster}}$, for the base rule, or distinct origins at their 99th percentile, and their thresholds, levels and known fleets are fitted on attack-free days. *In sample*, each calibration day is judged against a profile that includes it. *Cross-fitting*, built post hoc, judges each calibration day, like a test day, against a profile of the other days.

Five choices were made on the test days: counting origins, calibrating λ_e, the gate, the union with the unseen filter and the 5% fleet share. Of four shares, each added to the base rule, only 5% had no more false alarms than the rule alone on the first three test days, although 0.5% blocked about 25 times fewer legitimate clients and more of a 100-attacker botnet (Appendix E). The values $\rho = 3$ and $k_{\min} = 5$, the 99th percentile for τ_{cluster} and the 25-stack botnet were fixed on generated traffic before any production data were read.

We also report **collateral damage**, the share of legitimate traffic inside the scope: the false-positive rate of the filter an operator would deploy. On production traffic each configuration has a false-alarm rate (FA), the share of clean windows where it names a filter (5 643 on the test days and 1 152 on the held-out day, windows under $k_{\min}$ origins left out). Its exact 95% interval assumes independent windows, although false alarms cluster by day. Each alarm also has a collateral (Coll.), the share of that window's clients the filter blocks, given as a median over alarms. The rate times the mean collateral is the expected share of clients blocked per clean window.

The tables also report the rate of the scope without the gate (No gate, scope as trigger), the rate under a flash crowd of 1 000 users (Fl. 1k), and the blocked share of botnets sized at one median window or one tenth of it ($1\times$, $0.1\times$). This share is averaged over all attack windows, including those where the gate does not fire. It is the product of how often the gate fires, how often the scope names the botnet when it does, and the share of attackers the named filter covers.

V. RESULTS

We first show the mechanism on generated traffic. On production traffic, false alarms stay low but calibration sets a floor, the trigger decides what is stopped, and on 14 pre-specified days the rate doubles yet stays within the budget.

A. The Scope on Generated Traffic

On generated traffic, each scope flags the sessions it matches, needing neither training nor labels (Table III), while per-session detection stays at chance (Appendix C).

The obvious scope fails. With a single botnet stack, the modal fingerprint and the test agree, blocking 89.8% of the attack with no observed collateral. From five stacks onward, as Section III-C predicts, the modal fingerprint becomes a *legitimate* one: it blocks 0.0% of the attack and 39.0% of legitimate traffic, rising to 61.1% under a more concentrated legitimate population ($\alpha = 2.0$).

Profile-relative scopes succeed. The binomial test and the z-score block 90.0% of the attack at five stacks and 90.3% at 25, with **no collateral observed** across the campaigns. The rest sits on one-off fingerprints, so a scoped mitigation trades completeness for precision. At 100 stacks each stack holds about ten sessions, too few to stand out against a 1 000-session profile, so the test falls well below the z-score and the unseen filter (Table III). A profile pooled over 30 attack-free periods restores it to 89.6%. The unseen filter, with mean collateral 0.03% (at most 0.5% in one seed), matches or exceeds the test because new stacks never occur in generated legitimate traffic. It blocks none of the shared stacks, which real clients also present. On those the test blocks 85.4% at 2.23% collateral, and the z-score 89.9% at 3.37%.

The learned model needs labels unavailable to an operator during an attack, and at 25 stacks and zero false positives the test is well ahead. Given $\log b(f)$ and its log enrichment per session, the model recovers most of the attack at 25 and 100 stacks (Table III), but trains on labels from the campaign it scores. At 1% false positives, which the test does not need, (d) reaches 66.8% at 25 stacks and 94.6% with the profile.

Two conditions bound the result. When the botnet adopts common legitimate fingerprints, the boundary of the threat model, little is enriched. The test blocks 30.4% of the attack at 3.78% collateral, and the z-score 57.3% at 11.3%. The modal fingerprint blocks 3.6% at 39.0%, and the learned model 23.4% at 1% false positives. A flat or missing profile makes every fingerprint look rare and raises collateral to 77.6%, while a profile drifted to $\alpha = 2.0$ does little harm. Per window, $\Omega \geq \tau_{\text{cluster}}$ alone also fires on flash crowds [32]. The test removes those alarms for generated crowds (Appendix E), though not for crowds of production clients, as shown next.

B. Production Traffic: False Alarms and the Floor

False alarms. Pooled across endpoints, the calibrated scope and gate each exceed their 1% budget alone on the test days,

TABLE IV
THE CONFIGURATIONS ON THE TEST DAYS, ALL ENDPOINTS.

	Clean windows				New		Shared	
Config.	FA	No gate	Coll.	Fl. 1k	0.1×	1×	0.1×	1×
Test days, calibrated in sample								
binomial	0.1	2.2	32.9	22.2	3.0	19.9	1.9	17.6
beta-bin.*	0.4	4.4	10.5	28.4	3.3	31.4	2.7	24.7
z-score	0.4	6.1	10.8	13.1	3.1	28.8	2.3	18.9
*Test days, cross-fitted**								
binomial	0.1	1.9	31.4	20.7	3.0	19.8	1.5	17.4
beta-bin.*	0.1	1.1	2.0	10.9	3.0	23.3	2.1	17.5
z-score	0.2	2.5	11.0	5.5	3.0	20.1	2.0	16.0

In %. FA, No gate, Fl. 1k: clean windows. Coll.: alarms. New, Shared: blocked, attack windows. *Post hoc. 0.1×, 1×: botnet size, in median windows.

TABLE V
CALIBRATION FLOOR OF THE CONFIGURATIONS.

		λ_e		Binomial		Beta-binomial*	
Endpoint	n_0	bin.	beta*	new	shared	new	shared
E1, RUM beacons	2 983	10^{-21}	10^{-4}	139/139	254/286	85/139	86/167
E2, web console	62	10^{-73}	10^{-2}	139/139	1090/1041	56/114	84/168
E3, API	41	10^{-11}	10^{-2}	139/139	168/227	56/114	56/140
E4, SSO	20	10^{-4}	10^{-2}	84/84	84/114	56/56	84/84

n_0: median window. λ_e in sample. Floors in attackers per window, in sample / cross-fitted, test-day medians. *Post hoc, as is cross-fitting.

and only together stay within it (Table IV). The binomial configuration's scope alone names a filter in 2.2% of clean test-day windows, and the distinct-origin gate alone fires in 3.0%, reaching 5.4% on the busiest endpoint. Together they fire on only 0.1% of clean windows (5 of 5 643, 95% interval 0.03–0.21%), 4 of them on the web console. A fleet can open the gate while enriching its own fingerprint, yet the 5 joint misfires, versus 3.5 under independence, are within chance ($P = 0.27$, Appendix E). These misfires block a median 32.9% of the window's clients, and an expected 0.03% of clients per clean window. Crowds of 1 000 of the day's clients trigger the configuration in 22.2% of windows, with lighter filters blocking a median 4.3% of the window's clients.

What the scope can name: the calibration floor. The strict level that keeps false alarms rare also raises the floor. On the busiest endpoint, fleets push λ_e down to 10^{-60}, and exempting its five or six known fleets raises it to about 10^{-21}. For a 25-stack botnet on new stacks, the unseen filter sets the floor on E1 to E3: 139 attackers per window, so that the average stack reaches $k_{\min}$ origins. On E4 the test names the stacks first, from 84 (Table V). On stacks that real clients also present, mostly rare ones past the profile's ten most common fingerprints, the level sets the floor, computed at their median share. A typical stack is then named only past 254 attackers on the busiest endpoint, 8% of its window, and past 4 to 19 median windows on the small ones, as medians of daily ratios.

The floor grows with the number of stacks, from 2% of the busiest window at 5 stacks to 34% at 100, and with the fingerprints' popularity: those of rank 11 to 35 are never named on the two largest endpoints (Appendix E). It is no cliff: below it the scope names the stacks that chance makes larger, as a

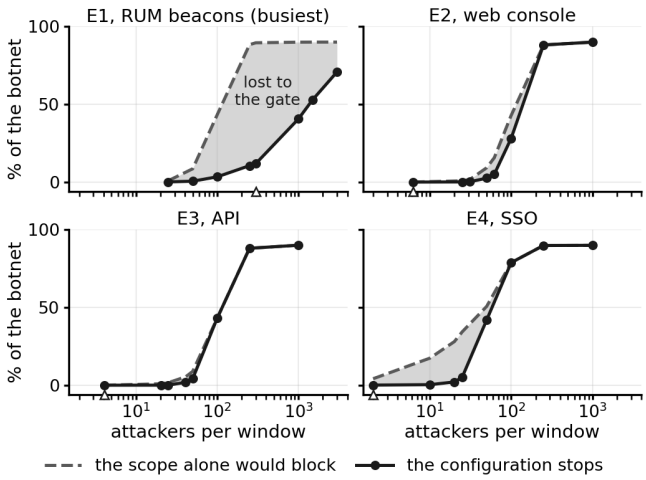


Fig. 2. What the binomial configuration stops, by the size of a 25-stack botnet on new stacks, test days. Dashed: what the scope alone would block in every window. Shaded: what the distinct-origin gate lets through, most of a small botnet on the busiest endpoint. Triangle: a tenth of the median window.

binomial model of stack sizes predicts (43.1% predicted versus 43.2% measured for 100 new-stack attackers on E1, Fig. 2).

Past the ratio limit the scope is expected to name nothing. The 4 to 28 most common fingerprints of each endpoint, which carry 86–94% of its origins, lie past that limit for a 25-stack botnet. A botnet using the endpoint's 25 most common fingerprints, the boundary of the threat model, is therefore mostly missed: the configuration stops 7.0% and 23.8% of 100 and 1 000 attackers. It stops none on the two largest endpoints, where nearly all 25 exceed the limit, but 73.5% of 1 000 on the single sign-on, where only 4 or 5 do.

C. What Is Stopped: The Trigger Decides

A named botnet is stopped only where the distinct-origin gate also fires (Fig. 2). On the busiest endpoint, the gate opens in only 6–13% of windows for botnets from 25 attackers up to a tenth of the median window. The binomial configuration therefore stops 3.4% of a 100-attacker botnet and 11.8% of a tenth-size botnet, 0.6–37.2% depending on the day. Its scope alone would block 43.2% and 89.6%. On the small endpoints, 100 attackers equal 1.6 to 5 median windows and open the gate in 66–100% of windows, leaving the floor as the main limit. A botnet of one median window opens it in only 14–33%.

Other triggers. To locate the loss, we examined two other triggers, both exploratory, after the held-out day was read (Table VI, with the binomial in sample, as the protocol fixed it). A *seasonal gate* divides the distinct origins by their median at the same hour of the calibration days and fires at that ratio's 99th percentile. It stops more of the tenth-size botnet on the busiest endpoint. Behind it, however, only the cross-fitted beta-binomial and the unseen filter stay within the budget on the console, where the binomial and the in-sample beta-binomial misfire in 2.0–2.6% of windows. The *scope as its own trigger* (No gate, dashed in Fig. 2) stops about 90% of that botnet on new stacks every day. With the binomial and known fleets, its

TABLE VI
THE SCOPES BEHIND EACH TRIGGER ON PRODUCTION TRAFFIC.

		FA, E1		FA, E2–E4		New		Shared		
Trigger	**Scope**	test	held	test	held	test	held	test	**Fl. 1k**	**Coll.**
Origin gate	binomial	0.00	0.00	0.28	0.69	11.8	0.0	7.7	22.2	32.9
	beta-bin.*†	0.14	0.00	0.28	0.00	11.9	0.0	8.3	10.9	2.0
	unseen	0.00	0.00	0.07	0.00	11.8	0.0	0.0	3.4	3.3
Seasonal gate*	binomial	0.00	0.00	2.64	0.69	20.4	15.3	14.4	22.3	35.2
	beta-bin.†	0.07	0.00	0.21	0.00	20.4	15.3	15.1	10.9	16.9
	unseen	0.00	0.00	0.00	0.00	20.4	15.3	0.0	3.4	–
No gate*	binomial	0.97	0.00	4.38	1.04	89.6	89.7	61.1	22.4	32.6
	beta-bin.†	0.90	1.74	1.46	1.39	89.7	90.1	65.7	11.0	6.6
	unseen	0.28	0.00	0.42	0.35	89.6	89.7	0.0	3.4	5.7

In %. Held: held-out day. No gate: scope as trigger. E2–E4: max. Fl. 1k, Coll.: pooled. New, Shared: E1's tenth-size botnet. †Cross-fitted. *Post hoc.

rate on the busiest endpoint sits at the budget, rising from no false alarm on the first two test days to 8 of 288 on the last.

What the unseen filter and the test add. The unseen filter covers new stacks of at least $k_{\min}$ origins, and the test adds shared stacks and, in some configurations, smaller new ones. On the larger new stacks, as in E1's tenth-size botnet, the unseen filter without the test does as well under every trigger, with fewer false alarms and flash-crowd firings, since it names each such stack. Its false alarms as its own trigger cluster by day (Appendix E). For new stacks too small for the filter, the cross-fitted beta-binomial adds on the small endpoints and, as its own trigger, on the busiest one too (58% versus the filter's 43% of 100 attackers), while the binomial adds only on the single sign-on. Behind the seasonal gate, the cross-fitted beta-binomial stops 53–84% of 100 new-stack attackers on the small endpoints but only 3.9% on the busiest one.

D. What the Evidence Supports

Post hoc analyses. Of the calibrated scopes used alone, only the cross-fitted beta-binomial nearly meets its budget (1.1%, Fig. 3). In sample the beta-binomial lowers the shared-stack floor on the three largest endpoints (Table V), but 19 of its 22 false alarms fall on the web console, where 74.4% of 1 000-user crowds trigger it. A first cross-fitting design, calibrated only on the last calibration day, was dropped after its results were seen because one day misses the rare fleet events that set the level. Under cross-fitting, the binomial configuration barely changes, with 6 false alarms versus 5, and the calibrated z-score loses its lead over the binomial (Fig. 3).

Prospectively, the rate doubles. The held-out day gave the binomial configuration 2 false alarms in 1 152 clean windows, consistent with the test days' rate ($P = 0.27$). Its gate opened in only 3 of them, and the scope misfired in two, both on the web console, more often than the test days' 5 of 168 openings predict (post hoc, $P = 0.003$). The 14-day block was exported after its protocol was public and analyzed after its export hashes were published (Appendix F). There the configuration misfired in 31 of 16 103 clean windows (0.19%, 95% interval 0.13–0.27%, Table VII). That is twice the test days' rate and, by the protocol's test, inconsistent with it ($P < 10^{-4}$), though within the budget. The gate opened half as often, but the scope misfired in 31 of its 233 openings (13%), versus 5 of 168 ($P < 10^{-11}$). The misfires fell on weekdays (30 of 31) and on the small endpoints, 17 of them in the console's 29 gate openings, and blocked a median 39% of the window's clients.

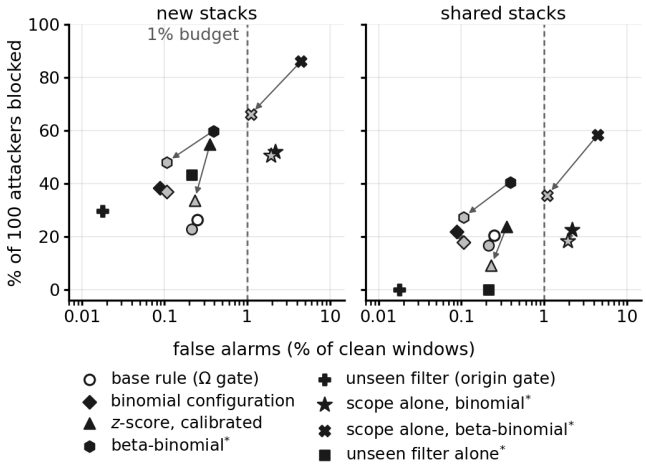


Fig. 3. Operating points on production traffic for 100 attackers per window on the test days, pooled over the endpoints, where the small ones dominate. Dashed line: the 1% budget. Gray: cross-fitted*, with an arrow from in sample where the two are apart. Alone: as its own trigger. *Post hoc.

The comparison fixed in advance prefers the seasonal gate. Behind it, the cross-fitted beta-binomial stays within the budget everywhere (0.05–0.35%) and is preferred to the unseen filter as its own trigger on every endpoint. On the small endpoints it stops 68–88% of 100 attackers on new stacks, versus 43%, and 34–66% on shared ones, which the filter misses. On the busiest endpoint it is preferred only for shared stacks, which the filter cannot name: it adds 11.4% of the tenth-size botnet there but stops only 11.8% on new ones, versus 89.6%. The unseen filter raised no false alarm, and the binomial scope with known fleets as its own trigger exceeded the budget there (1.56%).

This WAF's verdicts cannot serve as labels. The operator's WAF, the only source of labels at hand, blocked a median 9–63% of each window's clients on E2 to E4, and none on E1. A profile built without those clients makes the scopes agree with the WAF by construction: a stable fingerprint is then enriched only if 70–88% of its clients were blocked. With a profile of all clients, only 0.4–33% of the clients the three calibrated scopes would block were also blocked by the WAF, below a random pick (10–63%) on every endpoint. On the API and single sign-on, the scopes catch at most 2 of the WAF's 38 surges (Appendix E).

VI. DISCUSSION AND LIMITATIONS

Which configuration to recommend. Two limits decide per endpoint: the calibration floor and the trigger's sensitivity. On the three small endpoints the cross-fitted beta-binomial names shared stacks only past 3 to 4 median windows and new stacks from 2 to 3, so for smaller botnets the fallback is a rate limit or a challenge to every client. *(i) Now* our candidate is the cross-fitted beta-binomial behind the seasonal gate, with the fallback

TABLE VII
THE SIX CONFIGURATIONS FIXED IN ADVANCE, ON THE 14-DAY BLOCK.

		FA			E1, $0.1\times$		100 attackers			
Trigger	**Scope**	all	E1	E2–E4	New	Shared	New	Shared	**Fl. 1k**	**Coll.**
Origin gate	binomial	0.19	0.00	0.42	2.9	2.4	36.8	22.1	18.8	39.0
Ω gate	base rule	0.26	0.00	0.69	0.0	0.0	29.4	22.0	16.8	46.6
Origin gate	z-score	0.12	0.00	0.30	3.0	2.6	56.2	26.9	3.0	40.1
No gate	unseen	0.00	0.00	0.00	89.6	0.0	43.2	0.0	2.7	–
Seasonal gate	beta-bin.†	0.12	0.35	0.05	11.8	11.4	61.0	39.7	33.5	65.7
No gate	binomial^{a}	–	1.56	–	89.6	67.7	43.4	4.5	1.2	4.3

In %. 16 103 clean windows, each day calibrated on all earlier days. E2–E4: max. 100 attackers, Fl. 1k, Coll.: pooled. †Cross-fitted. ^{a}E1 alone.

below its floor. It was built post hoc, but the comparison fixed in advance preferred it on every endpoint. On the same days it misfired in fewer windows than the binomial configuration (20 versus 31, though 14 versus none on the busiest endpoint) and stopped more of 100 attackers. Crowds of 1 000 users trigger it in a third of windows, with light filters, and its misfires block a median two thirds of the window's clients. The fallback then acts on its gate's other clean openings, 1.5% of windows. The calibrated z-score misfired as often and fired on fewer crowds, but stopped less. On the busiest endpoint the beta-binomial gives up most new-stack detection, which the unseen filter as its own trigger keeps with no false alarm. Running both there is untested. *(ii) Next*, a longer evaluation, since no later day has tested this choice: the binomial configuration misfires on weekdays, which a gate calibrated by day of the week may absorb, and the beta-binomial's misfires cluster on two days of the busiest endpoint. Whether a campaign at the floor exhausts the service depends on spare capacity, which we did not measure.

What each evaluation shows. The generator isolates the mechanism under mimicry imposed by construction. Its campaigns, a third to half of the window, are easier than the tenth-size case where production shows the gate failing, and its new stacks never appear in legitimate traffic, which is why an unseen filter matches the scope there. Production traffic measures false alarms on real clients, and detection only for an injected botnet of one shape (25 stacks, new, shared but mostly rare, or the endpoint's most common), over 23 days of one operator's own endpoints, too few to calibrate a weekly cycle. The five choices of Section IV-B were made on the days they are reported on, so only the held-out day and the block test them prospectively, and cross-fitting removes only the in-sample fit of the levels. Appendix C lists further limits.

The scope needs the attack's fingerprint to differ. A bot outside the threat model that presents a common fingerprint, or a fresh one per device, defeats it (Section V), and an alarm then leads mostly to the fallback. Encrypted Client Hello [33] still exposes the inner *ClientHello* to a defender terminating TLS, as assumed here. Profiles must also remain current: a release that gives many users a new fingerprint would immediately trip the unseen filter, a case not observed in 23 days. We injected neither a flash crowd concentrated on one client type, the harder case, nor an attacker that shapes the profile or the fleet exemption. Profiles should therefore be rebuilt from windows where the scope names nothing, and persistent new fingerprints should enter them only through operator action.

What exchange loses, and what the ontology adds. Among the standards we examined, STIX 2.1, DOTS and Flowspec lack a JA4 property in their core vocabularies, while OCSF records JA4 fingerprints in network events and in the evidence of findings [34] but defines no filter or remediation over them. MISP's importer drops the fingerprints, which STIX 2.1 carries only in an extension the importer does not map to MISP's own JA4 object. A 2025 extension of the ACL model can match TLS client parameters but names no fingerprint, and DOTS, whose filters reuse that model, does not adopt it [35]. Blocking by fingerprint therefore needs a JA4 match in DOTS filters and a JA4 property in STIX. The ontology adds nothing to detection accuracy or the trigger. It is the deployable specification carried into the evidence chain and compiled into the log-store query, which reproduced exported address and /24-pair counts, and JA4 pair counts where the WAF blocked no client (Appendix F).

VII. CONCLUSION AND FUTURE WORK

We treated the scope of application-layer DDoS mitigation as a calibrated statistical decision over the TLS fingerprints of an endpoint's clients, specified in a session ontology. On generated stealthy campaigns, profile-relative scopes block 90% of a botnet on up to 25 new stacks without labels and with no observed collateral, while the alarm's most common fingerprint blocks users once the botnet spans several stacks. On a CDN operator's traffic, legitimate fleets raise a calibration floor: a 25-stack botnet on rare shared fingerprints is named by the binomial test only past 8% of the busiest endpoint's window and 4 to 19 median windows on the small ones.

The binomial configuration's false alarms stay at 0.1% of clean windows on the days used to choose it, though its scope alone exceeds its budget there (2.2%). On 14 later pre-specified days they double, still within budget, because the scope misfires in 13% of gate openings. On those days two of three post hoc configurations held their budget. On the busiest endpoint, the gate lets most of a small botnet through. The unseen filter as its own trigger stops 90% of it on new stacks with no false alarm, and there a cross-fitted beta-binomial behind a seasonal gate, preferred on every endpoint, adds shared stacks but loses most new ones. Next come weekday calibration, HTTP/2 attacks and a captured stealthy campaign.

ACKNOWLEDGMENT

We thank Azion Technologies (https://www.azion.com/) for authorizing the measurements and traffic statistics used here. The authors used an AI assistant (Claude, Anthropic) to draft and edit text in all sections, to extend the ontology and to write all code and figures. The authors reviewed and verified all content and take full responsibility for it.

REFERENCES

[1] J. Snellman and D. Iamartino, "How it works: The novel HTTP/2 'Rapid Reset' DDoS attack," Google Cloud Blog, Oct. 2023, accessed Sep. 25, 2026. [Online]. Available: https://cloud.google.com/blog/products/identity-security/how-it-works-the-novel-http2-rapid-reset-ddos-attack

[2] E. Cambiaso, G. Papaleo, G. Chiola, and M. Aiello, "Slow DoS attacks: Definition and categorisation," *International Journal of Trust Management in Computing and Communications*, vol. 1, no. 3–4, pp. 300–319, 2013.
[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
[4] C. Kemp, C. Calvert, T. M. Khoshgoftaar, and J. L. Leevy, "An approach to application-layer DoS detection," *Journal of Big Data*, vol. 10, no. 1, p. 22, 2023.
[5] M. Odusami, S. Misra, O. Abayomi-Alli, A. Abayomi-Alli, and L. Fernández-Sanz, "A survey and meta-analysis of application-layer distributed denial-of-service attack," *International Journal of Communication Systems*, vol. 33, no. 18, p. e4603, 2020.
[6] L. Belcastro, C. Carlucci, C. Cosentino, P. Liò, and F. Marozzo, "Enhancing network security using knowledge graphs and large language models for explainable threat detection," *Future Generation Computer Systems*, vol. 176, p. 108160, 2026.
[7] R. Mahajan, S. M. Bellovin, S. Floyd, J. Ioannidis, V. Paxson, and S. Shenker, "Controlling high bandwidth aggregates in the network," *ACM SIGCOMM Computer Communication Review*, vol. 32, no. 3, pp. 62–73, 2002.
[8] A. G. Alcoz, M. Strohmeier, V. Lenders, and L. Vanbever, "Aggregate-based congestion control for pulse-wave DDoS defense," in *Proc. ACM SIGCOMM*, 2022, pp. 693–706.
[9] F. Soldo, K. Argyraki, and A. Markopoulou, "Optimal source-based filtering of malicious traffic," *IEEE/ACM Transactions on Networking*, vol. 20, no. 2, pp. 381–395, 2012.
[10] M. Boucadair and T. Reddy.K, "Distributed Denial-of-Service Open Threat Signaling (DOTS) Data Channel Specification," Internet Engineering Task Force, RFC 8783, May 2020. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8783
[11] C. Loibl, S. Hares, R. Raszuk, D. McPherson, and M. Bacher, "Dissemination of Flow Specification Rules," Internet Engineering Task Force, RFC 8955, Dec. 2020. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8955
[12] C. Dietzel, M. Wichtlhuber, G. Smaragdakis, and A. Feldmann, "Stellar: Network attack mitigation using advanced blackholing," in *Proc. ACM International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, 2018, pp. 152–164.
[13] F. Silveira and C. Diot, "URCA: Pulling out anomalies by their root causes," in *Proc. IEEE INFOCOM*, 2010.
[14] D. Brauckhoff, X. Dimitropoulos, A. Wagner, and K. Salamatian, "Anomaly extraction in backbone networks using association rules," in *Proc. ACM Internet Measurement Conference (IMC)*, 2009, pp. 28–34.
[15] Z. Li, M. Sanghi, Y. Chen, M.-Y. Kao, and B. Chavez, "Hamsa: Fast signature generation for zero-day polymorphic worms with provable attack resilience," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2006.
[16] S. Kandula, D. Katabi, M. Jacob, and A. Berger, "Botz-4-Sale: Surviving organized DDoS attacks that mimic flash crowds," in *Proc. USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2005.
[17] R. Bharathi and R. Sukanesh, "A PCA based framework for detection of application layer DDoS attacks," *WSEAS Transactions on Information Science and Applications*, vol. 9, no. 12, pp. 389–398, 2012.
[18] A. Martinetti, "Advancing threat intelligence: JA4 fingerprints and inter-request signals," Cloudflare Blog, Aug. 2024, accessed Sep. 25, 2026. [Online]. Available: https://blog.cloudflare.com/ja4-signals/
[19] A. Lakhina, M. Crovella, and C. Diot, "Mining anomalies using traffic feature distributions," in *Proc. ACM SIGCOMM*, 2005, pp. 217–228.
[20] S. Ranjan, R. Swaminathan, M. Uysal, A. Nucci, and E. Knightly, "DDoS-Shield: DDoS-resilient scheduling to counter application layer attacks," *IEEE/ACM Transactions on Networking*, vol. 17, no. 1, pp. 26–39, 2009.
[21] G. Fernandes Jr., J. J. P. C. Rodrigues, and M. L. Proença Jr., "Autonomous profile-based anomaly detection system using principal component analysis and flow analysis," *Applied Soft Computing*, vol. 34, pp. 513–525, 2015.
[22] Y. Xie and S.-Z. Yu, "Monitoring the application-layer DDoS attacks for popular websites," *IEEE/ACM Transactions on Networking*, vol. 17, no. 1, pp. 15–25, 2009.
[23] P. E. Kaloroumakis and M. J. Smith, "Toward a knowledge graph of cybersecurity countermeasures," The MITRE Corporation, 2021, accessed Sep. 25, 2026. [Online]. Available: https://d3fend.mitre.org/resources/D3FEND.pdf
[24] K. Liu, F. Wang, Z. Ding, S. Liang, Z. Yu, and Y. Zhou, "Recent progress of using knowledge graph for cybersecurity," *Electronics*, vol. 11, no. 15, p. 2287, 2022.
[25] M. Husák, M. Čermák, T. Jirsík, and P. Čeleda, "HTTPS traffic analysis and client identification using passive SSL/TLS fingerprinting," *EURASIP Journal on Information Security*, vol. 2016, p. 6, 2016.
[26] B. Anderson and D. McGrew, "TLS beyond the browser: Combining end host and network data to understand application behavior," in *Proc. ACM Internet Measurement Conference (IMC)*, 2019, pp. 379–392.
[27] X. Li, B. Amin Azad, A. Rahmati, and N. Nikiforakis, "Good bot, bad bot: Characterizing automated browsing activity," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2021, pp. 1589–1605.
[28] J. Althouse, "JA4+ network fingerprinting," FoxIO Blog, Sep. 2023, accessed Sep. 25, 2026. [Online]. Available: https://blog.foxio.io/ja4+-network-fingerprinting
[29] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symp.*, 2017, pp. 1093–1110.
[30] S. Frolov and E. Wustrow, "The use of TLS in censorship circumvention," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2019.
[31] OASIS, "STIX version 2.1," OASIS Standard, 2021, accessed Sep. 25, 2026. [Online]. Available: https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html
[32] J. Jung, B. Krishnamurthy, and M. Rabinovich, "Flash crowds and denial of service attacks: Characterization and implications for CDNs and web sites," in *Proc. 11th Int. World Wide Web Conf. (WWW)*, 2002, pp. 293–304.
[33] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, "TLS Encrypted Client Hello," Internet Engineering Task Force, RFC 9849, Mar. 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9849
[34] OCSF Project, "Open cybersecurity schema framework (OCSF), version 1.4.0," GitHub repository, Jan. 2025, JA4 in network events since version 1.3.0 and in finding evidence since 1.4.0. Accessed Sep. 27, 2026. [Online]. Available: https://github.com/ocsf/ocsf-schema
[35] T. Reddy.K, D. Wing, and B. Anderson, "Manufacturer Usage Description (MUD) for TLS and DTLS Profiles for Internet of Things (IoT) Devices," Internet Engineering Task Force, RFC 9761, Apr. 2025. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9761
[36] M. Shafi, A. H. Lashkari, V. Rodriguez, and R. Nevo, "Toward generating a new cloud-based distributed denial of service (DDoS) dataset and cloud intrusion traffic characterization," *Information*, vol. 15, no. 4, p. 195, 2024.
[37] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.

APPENDIX A
SUB-RELATION SPECIFICATION

The rule's coordination mass is

$$\Omega(S) \;=\; \sum_{i \in \mathcal{R}_{\mathrm{rel}}} w_i \left| E_i(S) \right|, \tag{1}$$

where $E_i(S)$ is the set of pairs of distinct origins $\{o(s_a), o(s_b)\}$ linked by sub-relation i, with $o(s)$ the source address of session s. Each pair counts once, so one client's many connections cannot mimic coordination. The weight w_i follows the attacker's cost of breaking the signal, from the TLS fingerprint (1.0) through endpoint convergence (0.6) to network proximity (0.3), since carrier-grade NAT makes a shared /24 a weak discriminator, and calibration supports only the top of that order (Appendix B). For an equality sub-relation, $|E_i(S)| = \sum_k \binom{n_k}{2}$ over class sizes n_k in distinct

addresses, so Ω depends only on the class sizes returned by the query (Section III-E). The production gate and endpoint term add plain-HTTP clients to the n of Section III-B, which counts an address once per fingerprint. The compiled query counts each address once.

`TLSFingerprint:` exact equality of the full JA4, the relation exercised here. A near variant, *ja4_distance = 1*, would absorb version drift within one client library, but is not an equivalence relation and would require pair edges, so it is specified but not exercised. `ReusedIdentity`, `TemporalPattern` and `PayloadSignature`, also specified but not exercised, match a shared cookie, token or username, inter-arrival sequences within a DTW distance τ_{DTW}, and payload summaries above a cosine similarity τ_{payload}. `EndpointConvergence:` same endpoint, here a host or an address and port, with path-pattern normalization specified but not exercised. `NetworkProximity:` shared IPv4 /24 or IPv6 /48, with a shared ASN specified but not exercised.

Appendix B
Weight Calibration

On scenarios from earlier runs, a near-uniform benign vocabulary against a monolithic botnet, maximizing the per-session ROC AUC of $w_{\mathrm{tls}}\, z(\mathrm{ja4}) + w_{\mathrm{ep}}\, z(\text{endpoint conv.}) + w_{\mathrm{net}}\, z(\text{net prox.})$ over the grid $\{0.3 \ldots 1.0\}^3$ (125 combinations, 60 scenarios), yields $(1.0,\ 0.3,\ 0.3)$. Only the TLS fingerprint discriminates alone (AUC 0.93, versus 0.50 for endpoint convergence and 0.58 for network proximity), and every endpoint weight in the grid reaches the same optimal AUC of 0.943, as does the ontology's $(1.0,\ 0.6,\ 0.3)$. Thus only the top of the order is identified, and the per-window rule performs equally well with uniform weights (Appendix E). Request counts come from all $322\,658$ benign CICIDS2017 sessions and durations from the $40\,351$ with two or more requests. The default pool of $2\,000$ benign fingerprints follows the Zipf shape of the 495 measured at one point of presence and exceeds the 76–$1\,163$ observed per production endpoint. Popularity is measured per request, which if anything overstates concentration.

Appendix C
Additional Limitations

Instrumentation and deployment. Counting origins by source address undercounts users behind carrier-grade NAT and overcounts clients rotating IPv6 privacy addresses, whose natural unit is the /64. The production run is offline, over counts per endpoint, so a live CDN deployment still needs the partitioning of Appendix D, not implemented here.

Coverage. HTTP/2 attacks and other application protocols (DNS, gRPC) require sub-relations beyond the Slow HTTP DoS family over HTTP/1.1, and $\Omega(S)$ instantiates only the three sub-relations exposed by the generator and captures, leaving the others' weights untested. We found no independent capture of the stealthy regime: BCCC-cPacket-Cloud-DDoS-2024 criticizes the application-layer coverage of earlier datasets and provides 17 TCP-based DDoS scenarios [36].

Laboratory captures and KLAGE. CICIDS2017 [3] (Slowloris, Slowhttptest, HULK, GoldenEye) and CIC-IoT2023 [37] (DDoS Slowloris, HTTP Flood), on which KLAGE reports $F_1 = 0.841$ for `DDoS Slowloris`, run through the same extraction pipeline (PCAP $\rightarrow$ JA4 + flows $\rightarrow$ sessions). On CIC-IoT2023 `DDoS Slowloris`, whose attack clusters contain seven origins in one of 143 /24 prefixes, a strong per-session baseline using nine flow attributes reaches $F_1 = 0.900$ (AUC 0.987), while three of them plus cross-session evidence reach 0.911 (AUC 0.982). Real Slowloris, unlike our stealthy campaigns, leaves a per-session flow signature. The comparison with KLAGE is *uncontrolled*: our strong baseline also exceeds the published 0.841, and the released code starts from a graph of unpublished construction and provides no weights, preventing a controlled rerun.

Per-session detection and learned models. Each configuration, (a) to (d), runs on 30 seeds (Table III uses 15), with each campaign split 70/30 into training and test sessions. Configuration (a) uses only the nine per-session flow attributes in a Random Forest (RF), repeated with gradient boosting, a multilayer perceptron and logistic regression. Configuration (b) adds per-session ontological attributes, (c) adds only network proximity, and (d) adds cross-session evidence from the three sub-relations: the share of the cluster sharing one JA4 or /24 plus its size. We reimplement PCA profiling [21], PCA with k-means over a behavioral matrix [17] and a supervised RF/SVM over session features, after [4]. At $K = 50$ and $K = 1000$, (a) reaches a per-session ROC AUC of 0.498 and 0.503, (b) 0.500 and 0.502, the three academic baselines 0.495–0.518, and (a) stays at 0.488–0.503 under every classifier family ($\alpha = 1.5$, $M = 25$, 30 seeds), the chance level the stealth mode imposes. The cross-session representation (d) reaches 0.927 and 0.982, while (c) reaches 0.499 and 0.659 (paired Wilcoxon tests, Bonferroni-corrected, $p_{\mathrm{Bonf}} = 7.5 \times 10^{-9}$ for (d)$-$(c) and (d)$-$(a) at $K = 1000$).

Learned representation. That separability does not transfer. Fragmenting the botnet across M stacks makes cross-session evidence non-monotonic in the label: at $K = 1000$, an attacker shares a fingerprint with a few dozen peers, while a legitimate client in the benign head shares one with hundreds. Trained on five stacks and tested on 25 or 100, (d) falls to 0.61 and 0.63, and trained on 25 it falls to 0.48 on five and 0.75 on 100, versus 0.95–0.995 on the same M with other seeds. Cross-M tests use disjoint seeds because the generator draws every benign session before any M-dependent step: a same-seed split would score (a) at 0.88–0.94 by memorization.

Appendix D
Cost Model

Admission runs per request and the Ω aggregation once per window, so we time the two separately on a synthetic window: 30% of sessions in one campaign sharing a botnet JA4, benign JA4 from a pool of $2\,000$, nine endpoints, dispersed /24s. Each runs with pair edges, as the rules are written, and with class counts for the equality sub-relations (medians of three repeats on one core), and both give the same $\Omega(S)$ to within 10^{-11}.

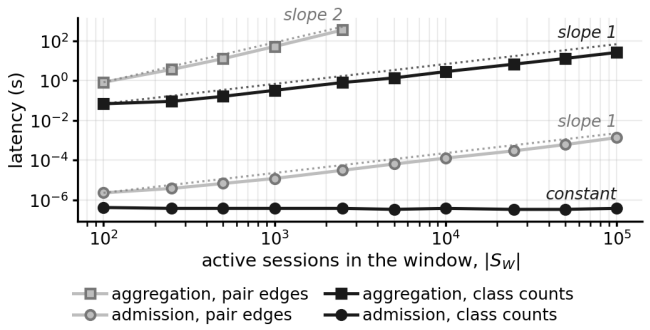


Fig. 4. Cost of admission, per request, and of the Ω aggregation, per window, with pair edges and with class counts. Dotted: reference slopes.

```
"@type": "kg:CoordinatedHTTPFlood",
"kg:clusterSize": 1991, "kg:coordinationScore": 1303422.8,
"kg:activatedSubRelations": [
 {"@type": "kg:relatedByTLSFingerprint",
  "kg:coordinationWeight": 1.0, "kg:linkedPairs": 114710},
 {"@type": "kg:relatedByEndpointConvergence",
  "kg:coordinationWeight": 0.6, "kg:linkedPairs": 1981045},
 {"@type": "kg:relatedByNetworkProximity",
  "kg:coordinationWeight": 0.3, "kg:linkedPairs": 286}],
"kg:derivedMitigationScope": {
  "kg:endpoint": "10.0.0.1:443",
  "kg:tlsJa4": ["t13d1516h2_synth_0001_00", ...]}
```

Listing 1. Evidence chain of the default campaign ($M = 25$, $K = 1000$), counted in origins, abridged. Endpoint pairs carry 91% of Ω, and the scope lists 25 fingerprints.

The symbolic layer uses `rdflib` instead of Jena/TDB2, so its times are likely an upper bound. With pair enumeration, median admission latency grows from 2.3 μs at $|S_W| = 100$ to 127 μs at 10 000, and with class counts it stays between 0.33 and 0.42 μs up to 100 000 sessions (Fig. 4). The pair-edge path takes 0.84 s at $|S_W| = 100$ and 52.8 s at 1 000. Class aggregation, with one membership triple per session and sub-relation plus a SPARQL query summing weighted class counts, takes 0.33 s at 1 000 sessions, 161 times faster, and 26.4 s at 100 000. The quadratic term therefore lies in the value of $\Omega(S)$, which counts pairs, and in pair edges of non-transitive sub-relations.

Partitioning at CDN scale. The rule reads one endpoint per S, so endpoint partitioning evaluates it *exactly* with no cross-shard traffic. A campaign spreading one JA4 across many endpoints then has too few pairs in any shard, and a global sketch, left unimplemented, would be needed to catch it.

Appendix E
The Rule per Window and Production Details

Here the rule runs as deployed, per $W = 300$ s window and endpoint, and no threshold sees an attack or label. τ_{cluster} is the 99th percentile of Ω over the windows of 30 attack-free runs, which also form the profile. False alarms are counted on 30 more runs, detection on default campaigns ($\alpha = 1.5$, $M = 25$) over windows with at least $k_{\min} = 5$ attackers, and a flash crowd retimes 25, 50 or 100 legitimate sessions from a third run into one window. We spread benign starts over 60 min as the generator otherwise packs them early.

Over 30 runs each, Ω passes τ_{cluster} in 77.8% and 85.7% of the 90 and 35 attack windows at $K = 1000$ and 50, in 0.6% of 360 clean windows and in 80–100% of the flash crowds of 25 to 100 users. A flash crowd adds mass as a campaign does. The test separates them because generated crowds draw users from the benign mix the profile describes, whereas a surge on one client type is enriched, as production fleets show. With the test, the rule fires on no clean window (one-sided 95% upper bound 0.8%) and no flash crowd, names a filter in every attack window flagged by Ω and blocks a median 75–80% of attackers, with no legitimate session hit. With uniform weights Ω catches at least as many attack windows (78.9% and 85.7%), and without its endpoint term 33.3% and none, while flagging 53–100% of flash crowds. A distinct-origin gate catches 82.2% and 85.7%, and with the test no clean window or flash crowd.

Production traffic. The exports contain no prefixes, so the scope is judged on JA4 alone. At the nominal level, it names a filter in 60.1% of clean windows when counted in connections, since one client opens many, and 26.9% when counted in origins, because legitimate fleets switch on together. Calibrating λ_e on the preceding days brings the base rule to 0.2%. The distinct-origin gate, which fires with Ω in 136 of the 181 clean windows where either fires, halves the base rule's false alarms with no loss of detection. At $\lambda_e = 10^{-60}$ the binomial test names a stack only past 15 to 18 origins. Fleets, such as a probe fleet active only in business hours, push λ_e down, and without fleets exempted the test alone names a filter in 1.5–6.3% of the next day's windows, often a fleet blocking about half of a window's clients.

Details of V-B. Windows with fewer than $k_{\min}$ origins, where no gate can open, are left out: 117 of E4's 1 440 test-day windows, none on the held-out day. In attack windows the calibrated scopes block a median 0% of legitimate clients and up to 71% in the worst window of a small endpoint. Gate and scope misfire together more often than independence predicts for the post hoc beta-binomial and the calibrated z-score, in 22 and 20 windows versus 7.5 and 11.9 ($P < 10^{-4}$ and $P = 0.02$ for a Poisson count), and in 6 and 13 versus 1.8 and 4.6 cross-fitted. For the binomial configuration the excess is within chance, 5 versus 3.5 and 6 versus 3.2 cross-fitted ($P = 0.27$ and 0.10). The expectation pools days per endpoint, so day-to-day variation alone can inflate the excess. The misfires of the other two also cluster by day, 13 of each on one day of the console, so exact intervals, which assume independent windows, understate their uncertainty. Under the binomial in sample, the shared-stack floor for 5 and 100 stacks is 50 and 1 026 attackers on E1, 164 and 6 158 on E2, 28 and 1 017 on E3 and 17 and 454 on E4. For 25 stacks, fingerprints of rank 11 to 35 need 808 on E3 and 114 on E4, and those of rank 36 to 100 need 743, 1 898, 227 and 84.

Details of V-C. Pooled, the binomial configuration stops 38.4% and 77.7% of 100 and 1 000 attackers on new stacks and 21.7% and 63.3% on shared ones, figures the small endpoints dominate. On the busiest endpoint, depending on the day, the gate opens in 0.7–42% of the windows of a botnet a tenth the size of the window. It opens in 45% at a third and in

79% at a whole window. Of 100 attackers on new stacks, the unseen filter by itself behind the origin gate stops 29.4%, the calibrated z-score 54.7% (23.7% on shared ones) and the post hoc beta-binomial 59.8% (40.5%). Fingerprints absent from the profile appear daily but rarely concentrate: only 0.21% of test-day windows contain one with at least $k_{\min}$ origins. The unseen filter's misfires as its own trigger cluster by day: all 4 on the busiest endpoint and both on the console fall on one day, and 5 of the API's 6 on another. On each small endpoint, every scope in Table VI stops 90.0% of 1 000 attackers on new stacks under every trigger, on both test and held-out days.

Details of V-D. On the block, the configuration misfired in 0 to 6 windows a day, never on the busiest endpoint, and the base rule fired on 30 of those 31 windows. Gate and scope misfired together in 31 windows versus 2.9 if independent, and the scope misfired in 10 of the API's 98 gate openings and 4 of the single sign-on's 95. Counting the test days' own uncertainty, a one-sided Fisher test gives $P = 0.07$ for the rate's rise and $P = 0.0002$ for the misfires once the gate opens. The z-score reference misfired less (20 versus 31), fired on fewer crowds and stopped more of 100 attackers. Crowds of 100 users trigger the configuration in 3.4% of windows, and its misfires block an expected 0.08% of clients per clean window. On the busiest endpoint the origin gate opens in 0.0–13.9% of a tenth-size botnet's windows, depending on the day, and the configuration stops 0.0–12.5% of it. Behind the seasonal gate, 12 of the beta-binomial's 14 misfires on the busiest endpoint fall on two days. No fingerprint absent from the profile reached $k_{\min}$ origins in a block window.

Post hoc, on ranks 36 to 100 the busiest endpoint's floor falls from 743 attackers under the binomial to 480 under the beta-binomial in sample, and rises to 949 cross-fitted. Compared with the binomial in sample, the cross-fitted beta-binomial's floor on those ranks is also higher on the API (255 versus 227), equal on the single sign-on and lower on the console (509 versus 1 898). The beta-binomial's levels then fall to 10^{-17}–10^{-2}, its scope alone fires in 1.2% of the held-out day's windows, and one fingerprint would be a known fleet on E1 on three of five days, though none qualifies in sample.

Most clients blocked by the WAF shared fingerprints with unblocked clients. With a profile built without blocked clients, 84–99% of the clients the scopes would block on the full traffic of the API and single sign-on were also blocked by the WAF. With a profile of all clients, the scopes catch the WAF's surges clearly above chance only on the console, and there with under 1% of blocked clients.

Other baselines. Uncalibrated and behind the Ω gate, the z-score fires on 2.6% of clean windows and 64.5% of 100-user flash crowds (86.8% of 1 000-user crowds).

Known fleets and the scope as a trigger. Of the four known-fleet shares, each added to the base rule on the first three test days, 5% was the only one whose false alarms did not exceed those of the rule alone (5 versus 6, the others 12 to 16). Using the rate times the median collateral, a rougher measure than the mean, the 0.5% share blocked about 25 times fewer legitimate clients per clean window on those days (0.0028% versus 0.070%). It also stopped more of a 100-attacker botnet on new stacks (44.6% versus 30.9%). Post hoc, the scope alone as a trigger, with known fleets, fires on 2.2% of pooled clean windows, versus 3.5% for the union without them. With known fleets it stops 61.1% of the busiest endpoint's tenth-size botnet on shared stacks, and its misfires there block a median 1.0% of the clients. The post hoc seasonal gate alone fires on 3.0% of clean test-day windows, as the origin gate does, and the binomial behind it stops 10.0–44.7% of E1's tenth-size botnet depending on the day. Setting $\rho = 2$ or 5 changes no pooled firing rate of the base rule by more than 0.3 points, no blocked share by more than 4.5 and no collateral median by more than 7.2.

Appendix F
Reproducibility

The reference implementation is available at https://github.com/Natanael0liveira/knowledge-graph-ddos-article under the MIT license. `make compile-check stix-ingest` verifies the compiled count query on generated sessions, also after a fourth equality signal is added as four triples. Exported bundles pass through the OASIS STIX 2.1 validator in strict mode, the reference TAXII 2.1 server (medallion 3.0.0) and MISP's importer (misp-stix 2026.9.16). The server returns them unchanged except for the extension definition it fails to serve. The compiled query reproduced exported address and /24-pair counts in every window of two production days, 1 152 windows each, and JA4 pair counts in the 288 windows of each day on the endpoint where the WAF blocked no client. Elsewhere the export drops clients partly blocked by the WAF, which the compiled query keeps. The protocol named JA4 class sizes, and the check compared their pair counts, which constrain the per-fingerprint counts the scope tests but do not identify them. The query ran in the operator's log store, the artifact keeps only its counts, and the evaluation reads the exports.

The held-out day's protocol was written, with export hashes, before the day was read (`fresh_day_protocol.md`), but committed 17 hours after the stated time, so no external timestamp attests that order. The block's protocol was pushed publicly before any of its days was exported, and the export hashes, with an addendum that adds three metrics, before any result was computed (`heldout_block_protocol.md` and its addenda). Its one amended rule, adopted after the export and before any result was computed, compares the small endpoints on 100 attackers and leaves the verdict unchanged. A few rows of raw counts were seen on screen at export. The repository holds every metric both protocols name, whatever its value (`production_tables.json`, `block_report.json`). Seeds are fixed, so every generated-traffic table and figure regenerates from its make target. Production tables (`make production-tables waf-labels`) regenerate from per-window results kept with the operator's data, while `make audit` recomputes about 250 printed numbers from anonymized summaries in the repository. We intend to submit the artifacts to the Artifact Evaluation track.